

Derin Takviyeli Öğrenme Yöntemleri ve Oyun Uygulamaları Üzerine Bir Literatür Taraması

Gizem Efe / Lotus AI Stajyeri

Derin Takviyeli Öğrenme (Deep Reinforcement Learning – DRL), takviyeli öğrenme algoritmalarının derin sinir ağları ile birleştirilmesiyle ortaya çıkan ve yüksek boyutlu, karmaşık problem uzaylarında etkili çözümler sunan bir yaklaşımdır. Bu çalışmada DRL'nin temel kavramları, öne çıkan algoritmaları ve özellikle oyun uygulamalarındaki kullanımı kapsamlı bir literatür taramasıyla inceledim. İncelemede başta **Deep Q-Network (DQN)** olmak üzere Aktör–Kritik yöntemleri ve politika tabanlı yaklaşımlar ele aldım; Atari oyunları, Snake oyunu ve Go gibi alanlarda elde edilen akademik sonuçlar değerlendirdim.

1. Giriş

Takviyeli öğrenme, bir ajanın çevresiyle etkileşime girerek deneme–yanılma yoluyla optimal davranışları öğrenmesini sağlayan bir makine öğrenmesi yaklaşımıdır [1]. Geleneksel takviyeli öğrenme algoritmaları küçük ve ayrık durum uzaylarında başarılı olsa da, durum sayısının artmasıyla birlikte ölçeklenebilirlik problemleri ortaya çıkmaktadır [1], [2].

Bu soruna çözüm olarak geliştirilen Derin Takviyeli Öğrenme, durum–eylem değer fonksiyonlarının veya politikaların derin sinir ağları aracılığıyla temsil edilmesini sağlar. DRL, özellikle oyun ortamlarında dikkat çekici başarılarla ulaşmıştır. Atari 2600 oyunlarında insan seviyesinde performans sergileyen ajanlar [3], Go oyununda dünya şampiyonlarını mağlup eden sistemler [4] bu yaklaşımın gücünü ortaya koymaktadır.

Bu çalışmanın amacı, DRL alanındaki temel kavramları, önemli algoritmaları ve oyun tabanlı uygulamaları literatür ışığında incelemektir.

2. Takviyeli Öğrenmenin Teorik Temelleri

Takviyeli öğrenme problemleri genellikle Markov Karar Süreci (Markov Decision Process – MDP) çerçevesinde tanımlanır. Bir MDP şu bileşenlerden oluşur [1]:

- **Durum kümesi (S)**
- **Eylem kümesi (A)**
- **Geçiş olasılıkları (P)**

- **Ödül fonksiyonu (R)**
- **İskonto faktörü (γ)**

Ajan, her zaman adımında bulunduğu durumu gözlemler, bir eylem seçer ve bu eylem sonucunda çevreden bir ödül alır. Amaç, uzun vadede beklenen toplam ödülü maksimize eden bir politika π öğrenmektir [1].

2.1 Değer Fonksiyonları

Takviyeli öğrenmede en önemli kavramlardan biri değer fonksiyonlarıdır.

- **Durum değeri:**

$$V^\pi(s) = \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t r_t \mid s_0 = s \right]$$

- **Durum-eylem değeri (Q-değeri):**

$$Q^\pi(s, a)$$

Q-değerleri, özellikle Q-learning gibi modelden bağımsız yöntemlerin temelini oluşturur [1].

3. Derin Takviyeli Öğrenme

Klasik Q-learning yöntemlerinde Q-değerleri bir tablo halinde tutulur. Ancak bu yaklaşım, durum uzayının büyümesiyle pratik olmaktan çıkar. Derin Takviyeli Öğrenme, bu tabloların yerini derin sinir ağlarının almasını sağlar [2].

Derin sinir ağları sayesinde:

- Yüksek boyutlu durumlar temsil edilebilir
- Ham piksel girdilerinden öğrenme mümkün olur
- Genelleme yeteneği artar

Bu yaklaşım, özellikle görsel girdilerin yoğun olduğu oyun ortamlarında büyük avantaj sağlar [3].

4. Deep Q-Network (DQN)

DQN algoritması, Q-learning yöntemini derin sinir ağı ile birleştirerek geliştirilmiştir [3]. Bu yöntemde $Q(s,a)$ fonksiyonu parametrik bir sinir ağı ile yaklaşık olarak hesaplanır.

4.1 Temel Yenilikler

DQN'in başarısının arkasındaki iki temel yenilik şunlardır:

- **Deneyim Tekrarı (Experience Replay):**

Ajanın yaşadığı deneyimler bir hafızada saklanır ve rastgele örneklenerek ağı eğitilir. Bu, örnekler arasındaki korelasyonu azaltır [3].

- **Hedef Ağ (Target Network):**

Q-hedeflerinin hesaplanmasında kullanılan ayrı bir ağı, eğitimi daha stabil hale getirir [3].

DQN, Atari 2600 oyunlarının büyük bölümünde insan seviyesinde ve üzerinde performans göstermiştir [3].

5. Aktör–Kritik ve Politika Tabanlı Yöntemler

DQN değer tabanlı bir yöntemdir. Bunun yanında, politika tabanlı ve hibrit yöntemler de literatürde önemli yer tutar.

5.1 Aktör–Kritik (A2C / A3C)

Aktör–Kritik yöntemlerinde iki bileşen bulunur:

- **Aktör:** Politikayı öğrenir
- **Kritik:** Değer fonksiyonunu tahmin eder

Asenkron Aktör–Kritik (A3C), birden fazla ajanın paralel olarak eğitilmesi fikrine dayanır ve öğrenme sürecini hızlandırır [5].

5.2 Proximal Policy Optimization (PPO)

PPO, politika güncellemelerini sınırlayan bir hedef fonksiyon kullanarak daha stabil öğrenme sağlar [6]. PPO, hem ayırık hem de sürekli eylem uzaylarında etkili olması nedeniyle günümüzde yaygın olarak tercih edilmektedir.

6. Oyun Uygulamaları

6.1 Atari Oyunları

Atari ortamları, DRL algoritmalarının karşılaştırılması için standart bir test alanı haline gelmiştir. DQN ve türevleri, yalnızca piksel girdileri kullanarak karmaşık oyun stratejileri öğrenmiştir [3].

6.2 Snake Oyunu

Snake oyunu, basit kurallara sahip olmasına rağmen gecikmeli ödül yapısı nedeniyle DRL için uygun bir problemdir. Literatürde DQN kullanılarak Snake oynayan ajanlar geliştirilmiş ve ödül fonksiyonunun öğrenme başarısı üzerindeki etkisi incelenmiştir [7].

6.3 Go ve AlphaGo

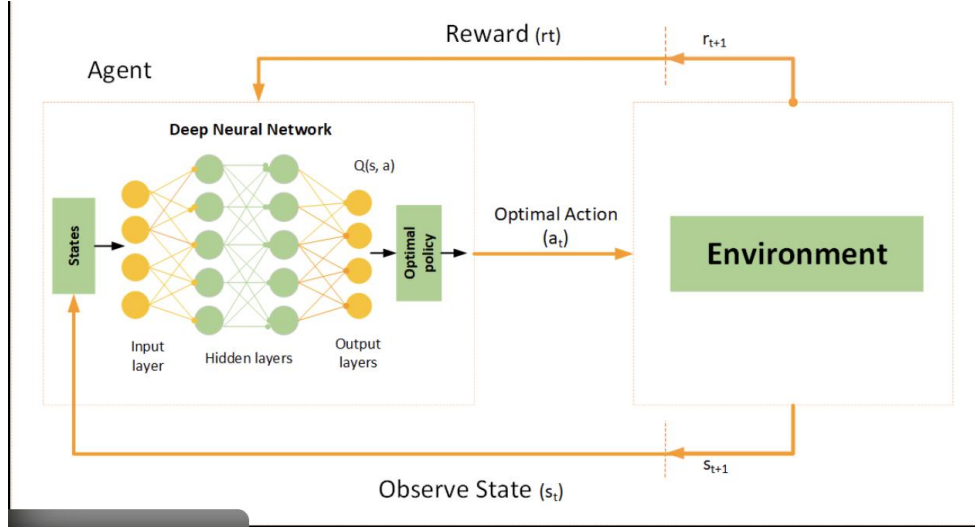
AlphaGo, derin sinir ağları ve ağaç arama yöntemlerini birleştirerek Go oyununda devrim yaratmıştır. Sistem, dünya şampiyonlarını mağlup ederek DRL'nin karmaşık strateji oyunlarındaki potansiyelini göstermiştir [4].

7. Snake Oyunu ve Uygulaması

7.1 Snake Oyunu İçin Deep Q-Network (DQN) Algoritmasının Seçilme Gerekçesi

Snake oyunu, basit kurallara sahip olmasına rağmen takviyeli öğrenme açısından çeşitli zorluklar barındıran bir problemdir. Oyunda ajan, her zaman adımında sınırlı sayıda eylem (yukarı, aşağı, sağ, sol) arasından birini seçmekte ve bu seçimlerin uzun vadeli sonuçlarını doğrudan gözlemleyememektedir. Bu özellikler, Snake oyununu ayırık eylem uzayına sahip, gecikmeli ödül yapılı bir problem haline getirmektedir.

Bu çalışmada Snake oyunu için Deep Q-Network (DQN) algoritmasının tercih edilmesinin başlıca nedenleri aşağıda açıklanmıştır.



Şekil 1 Snake oyunu için kullanılan Deep Q-Network mimarisi

7.2 Ayırık Eylem Uzayı ile Uyum

Snake oyununda eylem uzayı sınırlı ve ayırık yapıdadır. Ajan yalnızca dört yönlü hareket edebilmektedir. DQN, ayırık eylem uzaylarında doğrudan Q-değerlerini tahmin edebilen bir yöntem olması nedeniyle bu tür problemler için doğal bir seçimdir [3]. Politika tabanlı yöntemler veya sürekli eylem uzayı için geliştirilen algoritmalar (ör. DDPG) bu problem için gereksiz karmaşıklık yaratmaktadır.

7.3 Durum Uzayının Tablo Tabanlı Yöntemler İçin Büyük Olması

Snake oyununda durum uzayı; yılanın konumu, yönü, kuyruğunun şekli ve yemin yeri gibi birçok değişkeni içermektedir. Bu durum, klasik Q-learning yöntemlerinde kullanılan Q-table yaklaşımını pratik olmaktan çıkarmaktadır. DQN, Q-değerlerini bir sinir ağı ile yaklaşık olarak hesapladığı için yüksek boyutlu durum uzaylarında ölçeklenebilir bir çözüm sunar [1], [3].

7.4 Ödül Yapısının DQN ile Etkili Şekilde Öğrenilebilmesi

Snake oyununda ödüller genellikle seyreklerdir. Ajan, yalnızca yem yediğinde pozitif ödül almakta; çarpışma durumunda ise oyunu kaybetmektedir. Bu tür gecikmeli ve seyrek ödül yapıları, Q-değer tahmini yapan algoritmalar için uygundur. DQN, uzun vadeli ödülleri Bellman denklemi aracılığıyla yayarak öğrenebildiğinden, yılanın yalnızca anlık kazançlara değil, uzun vadeli hayatta kalmaya yönelik stratejiler geliştirmesini sağlar [1].

7.5 Literatürde Kanıtlanmış Başarı

DQN, Atari 2600 oyunlarında yalnızca ham piksel girdileriyle insan seviyesinde performans göstermiştir [3]. Benzer şekilde, literatürde Snake oyunu üzerinde yapılan çalışmalarda da DQN

tabanlı ajanların etkili sonuçlar verdiği rapor edilmiştir [7]. Bu durum, DQN'in oyun tabanlı problemlerde güvenilir ve yaygın olarak kabul edilen bir yöntem olduğunu göstermektedir.

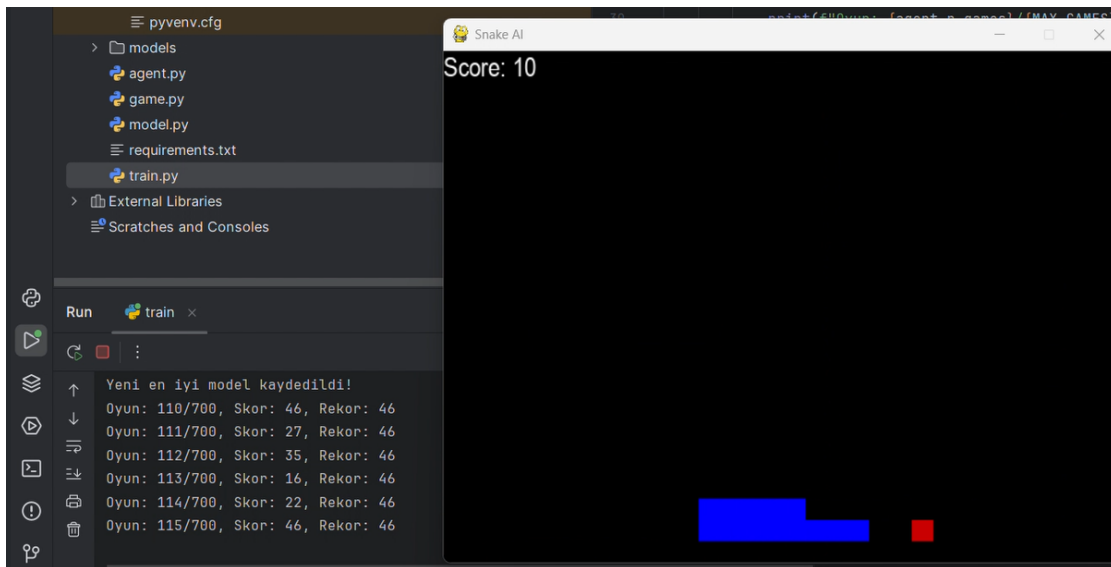
7.6 Uygulama ve Eğitim Kolaylığı

DQN algoritması, Aktör–Kritik veya PPO gibi yöntemlere kıyasla daha sade bir mimariye sahiptir. Tek bir ana ağ ve bir hedef ağ yapısı ile uygulanabilir olması, lisans seviyesindeki projeler için hem anlaşılabilir hem de geliştirilebilir bir yapı sunar. Ayrıca DQN'in açık kaynaklı birçok örneği ve dokümantasyonu bulunması, uygulama sürecini kolaylaştırmaktadır [2].

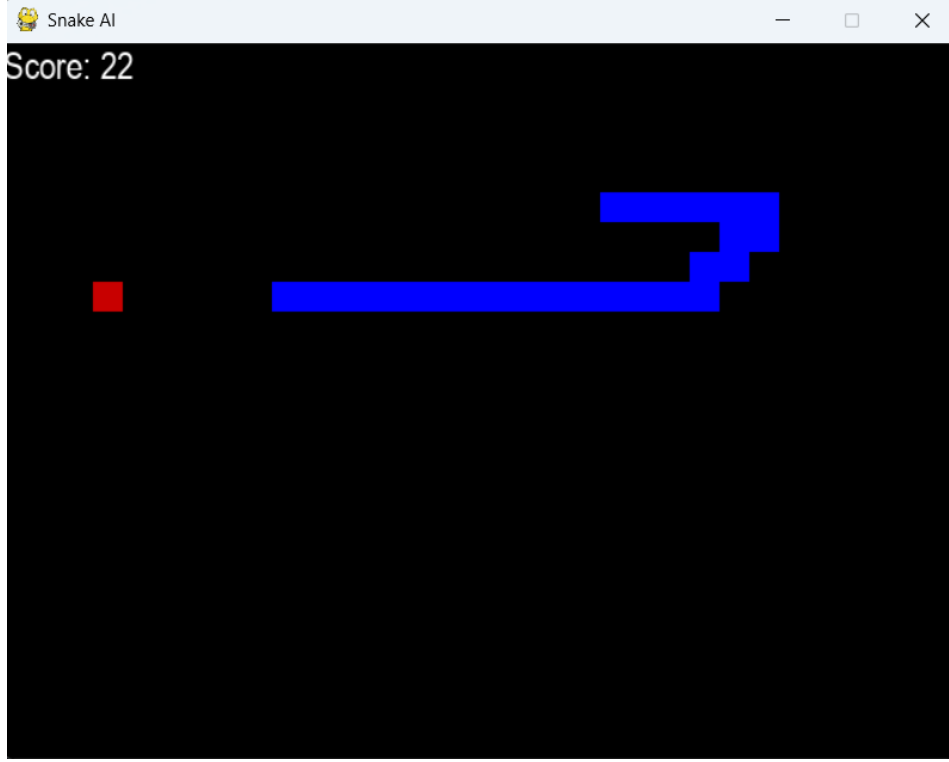
7.7 Eğitim Süreci ve Uygulama Detayları

Bu çalışmada Snake ajanı toplam 700 oyun (episode) boyunca eğitilmiştir. Eğitim sürecinde ϵ -greedy politika kullanılarak ajan hem çevreyi keşfetmiş hem de öğrendiği politikayı uygulamıştır. Eğitim sırasında elde edilen en iyi performansa sahip model ağırlıkları diske kaydedilmiştir. Eğitim tamamlandıktan sonra keşif mekanizması devre dışı bırakılarak ajan test modunda çalıştırılmıştır.

Test aşamasında ajan sırasıyla 26, 53, 44, 14, 43 ve 17 skorlarına ulaşmıştır. Elde edilen sonuçlar, ajanın rastgele hareket etmediğini ve öğrenilen politikanın genel olarak kararlı bir davranış sergilediğini göstermektedir. Skorlar arasındaki değişkenlik, yemin rastgele konumlandırılması ve oyunun stokastik yapısından kaynaklanmaktadır. Eğitim süreci boyunca elde edilen maksimum skorun 80 olması, DQN algoritmasının Snake oyunu için etkili bir öğrenme sağladığını ortaya koymaktadır.



Şekil 2 Eğitim sırasında DQN ajanının oyun sırasında sergilediği davranış



Şekil 3 DQN ajanı tarafından oynanan Snake oyunu ortamı

8. Tartışma

Literatür incelendiğinde, Derin Takviyeli Öğrenme (Deep Reinforcement Learning – DRL) yöntemlerinin özellikle oyun ortamlarında başarılı sonuçlar ürettiği görülmektedir. Bununla birlikte, bu yöntemlerin önemli dezavantajları da bulunmaktadır. DRL algoritmaları genellikle yüksek hesaplama maliyeti gerektirir ve anlamlı bir politika öğrenebilmek için çok sayıda bölüm (episode) boyunca eğitim yapılması gerekir. Bu durum, özellikle sınırlı donanım kaynaklarına sahip ortamlarda eğitim süresini uzatabilmektedir [1].

Bu çalışmada geliştirilen Snake oyunu uygulamasında da benzer gözlemler elde edilmiştir. Eğitim süreci ilerledikçe ajan başlangıçta rastgele hareketler sergilerken, zamanla yemlere daha bilinçli yönelme ve çarpışmalardan kaçınma davranışları geliştirmiştir. Ancak eğitim sürecinde skorların her zaman artan bir eğilim göstermemesi, takviyeli öğrenmenin doğası gereği öğrenmenin dalgalı bir yapı sergilediğini ortaya koymaktadır. Bu durum, literatürde de sıklıkla vurgulanan bir olgudur [6].

Ödül fonksiyonunun tasarımı, Snake oyunu özelinde öğrenme performansını doğrudan etkilemiştir. Yemin yenmesi durumunda pozitif, çarpışma halinde ise negatif ödül verilmesi ajanın temel hedefleri öğrenmesini sağlamıştır. Ancak ödül yapısının yeterince dengeli

olmaması durumunda ajan, kısa vadeli kazançlara yönelerek uzun vadede başarısız stratejiler geliştirebilmektedir. Bu nedenle ödül fonksiyonunun doğru ve problemle uyumlu biçimde tanımlanması kritik bir unsurdur [1].

Ayrıca epsilon-greedy keşif stratejisinin eğitim sürecindeki rolü de dikkat çekicidir. Eğitim ilerledikçe keşif oranının azaltılması, ajanın öğrendiği bilgileri daha tutarlı biçimde kullanmasını sağlamış; test aşamasında ise tamamen greedy bir politika ile oynandığında daha istikrarlı davranışlar gözlemlenmiştir.

9. Sonuç

Bu çalışmada, Derin Takviyeli Öğrenme alanındaki temel yaklaşımlar ve bu yaklaşımların oyun uygulamalarındaki kullanımı literatür ışığında incelenmiştir. DQN, Aktör–Kritik ve PPO gibi algoritmaların özellikle oyun tabanlı problemlerde etkili çözümler sunduğu görülmüştür. Bu yöntemler arasında DQN algoritması, ayrık eylem uzayına sahip olması ve uygulama kolaylığı nedeniyle Snake oyunu için tercih edilmiştir.

Gerçekleştirilen uygulama kapsamında, Snake oyunu ortamı Python dili kullanılarak modellenmiş ve bir DQN ajanı bu ortamda eğitilmiştir. Eğitim süreci sonucunda ajan, başlangıçtaki rastgele davranışlarını zamanla geliştirerek yemlere yönelme ve engellerden kaçınma yetenekleri kazanmıştır. Elde edilen skorlar, modelin anlamlı bir politika öğrenebildiğini göstermektedir.

Snake oyunu gibi nispeten basit yapıları ortamlar, DRL algoritmalarının davranış öğrenme kabiliyetlerini gözlemlemek ve temel prensiplerini anlamak açısından etkili bir test alanı sunmaktadır. Bu tür uygulamalar, lisans seviyesindeki çalışmalar için hem öğretici hem de geliştirilebilir projeler olarak öne çıkmaktadır.

Gelecek çalışmalarda, hedef ağ (target network) yapısının eklenmesi, ödül fonksiyonunun daha ayrıntılı tasarlanması ve PPO gibi alternatif algoritmalarla karşılaştırmalı deneyler yapılması, elde edilen sonuçların daha da iyileştirilmesine katkı sağlayabilir.

Kaynakça

- [1] R. S. Sutton, A. G. Barto, *Reinforcement Learning: An Introduction*, MIT Press, 2018.
- [2] Y. Li, “Deep Reinforcement Learning: An Overview,” *arXiv:1701.07274*, 2017.
- [3] V. Mnih et al., “Human-level control through deep reinforcement learning,” *Nature*, 2015.
- [4] D. Silver et al., “Mastering the game of Go with deep neural networks and tree search,” *Nature*, 2016.
- [5] V. Mnih et al., “Asynchronous Methods for Deep Reinforcement Learning,” *ICML*, 2016.
- [6] J. Schulman et al., “Proximal Policy Optimization Algorithms,” *arXiv:1707.06347*, 2017.
- [7] Y. Pan et al., “Playing the Snake Game with Reinforcement Learning,” 2023.